

Universidad Tecnológica del Centro de Veracruz

IDGS

Extracción del Conocimiento de Base de Datos

Creación e Implementación de un Data Warehouse usando PostgreSQL y Jupyter con Python

Producto 1er Parcial

Alumno:

Vichique Hernandez Eduardo Adan

Matricula

20223L001149

Docente:

María Reina Zarate Nava

Grupo:

9A

Junio 2025

PARTE I

Investigación Teórica

1. ¿Qué es un Data Warehouse?

Un Data Warehouse (DW), o Almacén de Datos, es una base de datos diseñada específicamente para soportar la toma de decisiones empresariales. A diferencia de las bases de datos operacionales, un DW integra, consolida y almacena grandes volúmenes de datos históricos provenientes de múltiples fuentes, optimizado para consultas analíticas complejas y reportes de negocio.

El concepto fue formalizado por Bill Inmon en 1990, quien lo definió como: "una colección de datos orientada a temas, integrada, no volátil y variante en el tiempo, que apoya la toma de decisiones gerenciales". Posteriormente, Ralph Kimball popularizó el enfoque dimensional (esquema estrella) que sigue siendo el más utilizado en la industria.

1.1 Características fundamentales

- **Orientado a temas:** Los datos se organizan en torno a los principales ámbitos del negocio (ventas, clientes, productos, inventario) en lugar de aplicaciones. Esto facilita el análisis por área funcional.
- **Integrado:** Consolida datos de múltiples fuentes heterogéneas —ERPs, CRMs, archivos planos, APIs— resolviendo inconsistencias de formato, nomenclatura y codificación.
- **No volátil:** Una vez cargados, los datos no se modifican ni eliminan; solo se agregan nuevos registros históricos. Garantiza la trazabilidad de la información.
- **Variante en el tiempo:** Mantiene el historial completo, permitiendo análisis de tendencias y comparaciones a lo largo de semanas, meses o años.

1.2 Componentes de un Data Warehouse

- **Fuentes de datos:** Sistemas transaccionales (OLTP), archivos CSV/Excel, servicios web, sensores IoT.
- **Capa ETL:** Extract (extracción), Transform (transformación y limpieza) y Load (carga). Es el corazón del DW.
- **Almacén central:** La base de datos relacional o columnar donde residen los datos integrados.
- **Data Marts:** Subconjuntos del DW orientados a departamentos específicos (ventas, finanzas, RRHH).
- **Capa de presentación:** Herramientas BI, dashboards, reportes y notebooks para análisis.

2. Diferencias: Base de Datos Operacional vs Data Warehouse

Las bases de datos operacionales (OLTP — Online Transaction Processing) y los Data Warehouses (OLAP — Online Analytical Processing) tienen propósitos, arquitecturas y comportamientos fundamentalmente distintos:

Característica	BD Operacional (OLTP)	Data Warehouse (OLAP)
Propósito	Soporte a operaciones diarias	Análisis y toma de decisiones
Tipo de datos	Actuales y detallados	Históricos y consolidados
Operaciones	INSERT, UPDATE, DELETE	SELECT con GROUP BY complejos
Usuarios	Personal operativo (miles)	Analistas y directivos
Diseño	Normalizado (3FN)	Desnormalizado (Star/Snowflake)
Actualización	Tiempo real continuo	Carga periódica (ETL)
Tamaño típico	MB a pocos GB	GB a TB o más
Tiempo de respuesta	Milisegundos	Segundos a minutos
Ejemplo	Sistema de punto de venta	Análisis de ventas históricas

3. Ventajas del uso de un Data Warehouse

- **Mejor toma de decisiones:** Permite acceder a datos históricos consolidados que revelan tendencias y patrones de negocio imposibles de detectar en sistemas transaccionales.
- **Rendimiento analítico mejorado:** Las consultas analíticas complejas se ejecutan sobre el DW sin afectar el rendimiento de los sistemas operativos.
- **Integración de múltiples fuentes:** Centraliza información de distintos sistemas (ERP, CRM, ventas, logística) en un repositorio único y consistente.
- **Calidad de datos garantizada:** El proceso ETL elimina duplicados, corrige inconsistencias y unifica formatos, entregando datos limpios y confiables.
- **Historial completo:** Almacena años o décadas de datos, permitiendo análisis de tendencias a largo plazo y auditorías.
- **Soporte a KPIs e indicadores:** Facilita la creación de dashboards, cuadros de mando e informes ejecutivos en tiempo reducido.
- **Escalabilidad:** Diseñado para crecer conforme aumenta el volumen de datos sin comprometer la disponibilidad del sistema.
- **Base para Business Intelligence:** Es el fundamento sobre el cual operan herramientas BI como Power BI, Tableau, Metabase o Apache Superset.

4. Arquitecturas Comunes de Data Warehouse

4.1 Esquema Estrella (Star Schema)

Es la arquitectura más común y sencilla. Consiste en una tabla de hechos central conectada directamente a múltiples tablas de dimensión desnormalizadas. Su estructura visual recuerda a una estrella, de ahí su nombre.

La tabla de hechos almacena las métricas cuantificables del negocio (ventas, cantidades, costos) junto con las claves foráneas que apuntan a cada dimensión. Las tablas de dimensión contienen los atributos descriptivos que contextualizan los hechos.

- **Ventajas:** Consultas simples con pocos JOINS, excelente rendimiento en herramientas BI, fácil comprensión para usuarios de negocio.
- **Desventajas:** Cierta redundancia de datos en las dimensiones desnormalizadas, mayor consumo de almacenamiento.
- **Ejemplo de este proyecto:** fact_ventas conectada a dim_cliente, dim_producto, dim_tiempo y dim_region.

4.2 Esquema Copo de Nieve (Snowflake Schema)

Extensión del esquema estrella donde las tablas de dimensión se normalizan subdividiéndose en sub-dimensiones. Visualmente asemeja un copo de nieve por las ramas que se generan.

- **Ventajas:** Reduce la redundancia de datos, ahorra espacio de almacenamiento, mayor integridad referencial.
- **Desventajas:** Consultas más complejas (más JOINS), generalmente menor rendimiento en operaciones analíticas.
- **Ejemplo:** dim_producto → dim_subcategoria → dim_categoria → dim_departamento.

4.3 Constelación de Hechos (Galaxy Schema / Fact Constellation)

Contiene múltiples tablas de hechos que comparten algunas tablas de dimensión. Es el esquema más complejo y se usa en entornos empresariales avanzados que modelan varios procesos de negocio relacionados.

- **Ventajas:** Mayor flexibilidad, permite modelar múltiples procesos de negocio (ventas, inventario, compras) en un mismo DW.
- **Desventajas:** Mayor complejidad de diseño, mantenimiento y comprensión para usuarios no técnicos.
- **Ejemplo:** fact_ventas y fact_inventario compartiendo dim_producto, dim_tiempo y dim_region.

5. Herramientas y Tecnologías para implementar un Data Warehouse

5.1 Bases de Datos

- **PostgreSQL:** RDBMS de código abierto, robusto y escalable. Soporte nativo para consultas analíticas complejas, particionado de tablas y múltiples extensiones. Elegido para este proyecto.

- **Amazon Redshift:** Data Warehouse columnar en la nube de AWS, optimizado para análisis de petabytes.
- **Google BigQuery:** DW serverless de Google Cloud con escalado automático y pago por consulta ejecutada.
- **Snowflake:** Plataforma cloud-native con separación de almacenamiento y cómputo, muy popular en empresas.
- **Microsoft Azure Synapse:** Servicio integrado de analítica de Azure con soporte SQL y Apache Spark.

5.2 Herramientas ETL

- **Python (pandas + SQLAlchemy):** Opción flexible y gratuita para ETL personalizado. Utilizada en este proyecto para extracción, transformación y carga de datos.
- **Apache Airflow:** Orquestador de flujos ETL basado en DAGs (Directed Acyclic Graphs), muy usado en producción.
- **dbt (Data Build Tool):** Herramienta moderna para transformación de datos directamente con SQL, con control de versiones.
- **Talend:** Suite ETL empresarial con interfaz visual de arrastrar y soltar, ampliamente usado en grandes empresas.

5.3 Herramientas de Análisis y Visualización

- **Jupyter Notebook/Lab:** Entorno interactivo para análisis con Python, SQL y visualizaciones integradas. Usado en este proyecto.
- **Matplotlib y Seaborn:** Librerías Python para generación de gráficos estadísticos y visualizaciones de datos. Usadas en este proyecto.
- **Power BI:** Herramienta de Microsoft para dashboards empresariales interactivos y reportes ejecutivos.
- **Tableau:** Plataforma líder de visualización de datos, reconocida por su facilidad de uso y potencia visual.
- **Apache Superset:** Plataforma BI de código abierto compatible con PostgreSQL, ideal para equipos técnicos.

PARTE II

Instalación y Configuración del Entorno

6. Instalación de PostgreSQL

6.1 Descarga e instalación

Para instalar PostgreSQL en Windows, se accede al sitio oficial <https://www.postgresql.org/download/> y se descarga el instalador para Windows. Durante la instalación se configura:

- **Puerto de conexión:** 5432 (puerto por defecto).
- **Usuario administrador:** "postgres" con contraseña definida por el alumno.
- **Stack Builder:** Opcional; se puede omitir al finalizar.

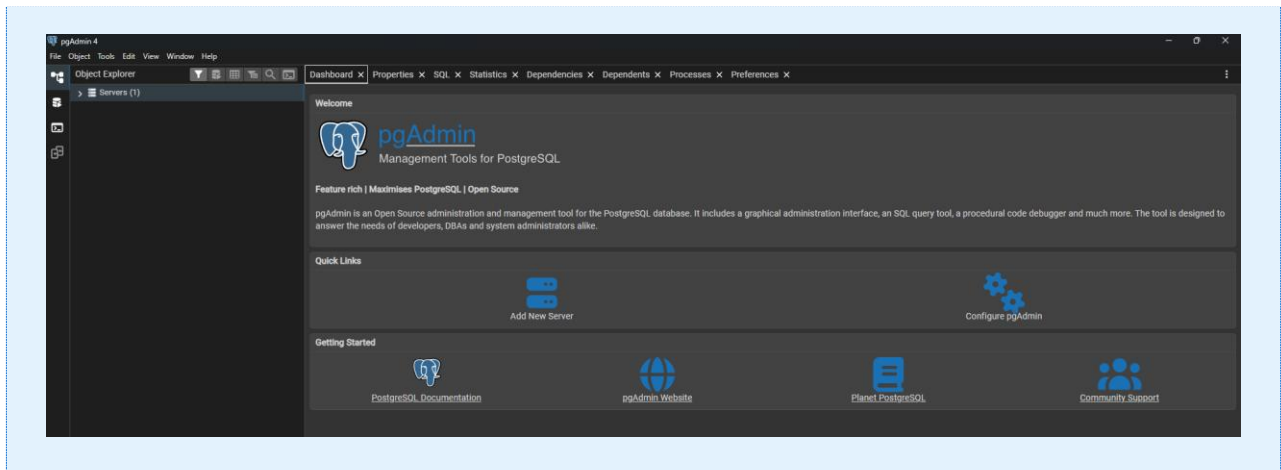


Figura: Instalación exitosa de PostgreSQL en el equipo local.

6.2 Verificación de la instalación

Al finalizar la instalación, se verifica que el servicio esté activo desde el Administrador de Servicios de Windows o ejecutando el siguiente comando en CMD/PowerShell:

```
pg_isready -U postgres
```

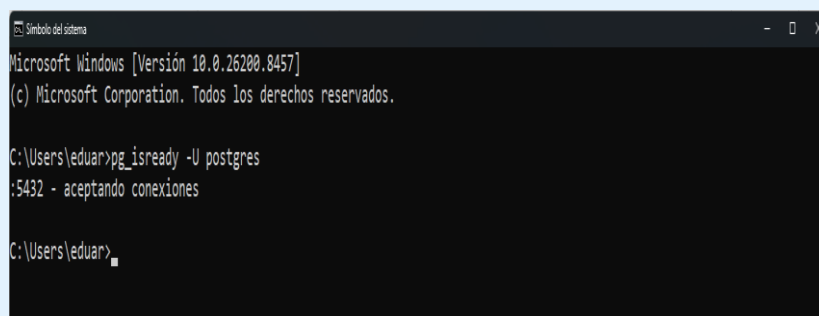


Figura: Servicio de PostgreSQL activo y escuchando en el puerto 5432.

6.3 Conexión con pgAdmin 4

pgAdmin 4 es la herramienta gráfica incluida con PostgreSQL para administrar las bases de datos. Se abre desde el menú de inicio y se conecta al servidor local con las credenciales configuradas durante la instalación.

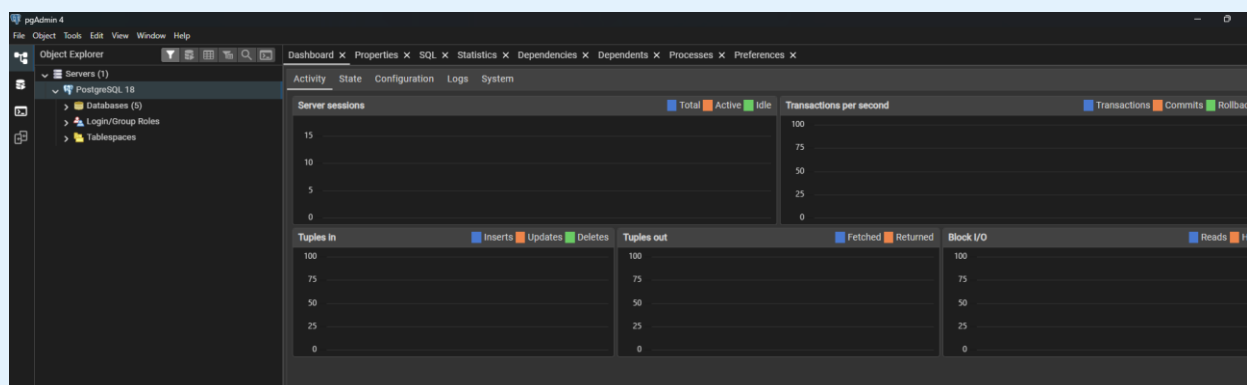


Figura: pgAdmin 4 conectado al servidor PostgreSQL local.

7. Creación de la Base de Datos del Data Warehouse

7.1 Creación de la base de datos dw_ventas

La base de datos del Data Warehouse se crea ejecutando el siguiente comando SQL en pgAdmin (Query Tool) o desde la terminal psql:

```
CREATE DATABASE dw_ventas;
```

Alternativamente, el Jupyter Notebook crea automáticamente la base de datos al ejecutar la celda de configuración de conexión, verificando si ya existe antes de crearla.

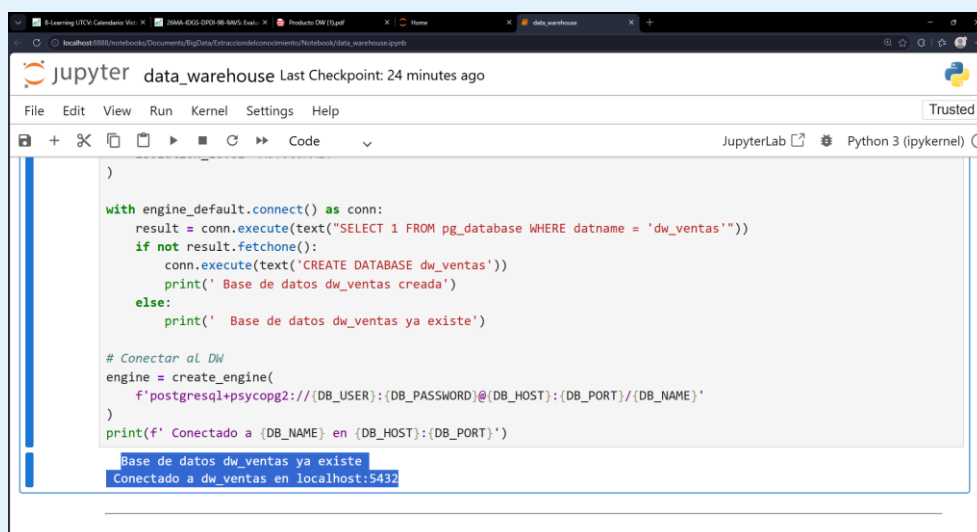


Figura: Base de datos dw_ventas creada exitosamente en PostgreSQL.

8. Diseño del Modelo Dimensional

8.1 Diagrama del Esquema Estrella

El Data Warehouse implementado utiliza un esquema estrella con una tabla de hechos central y cuatro tablas de dimensión:

Tabla	Descripción
dim_tiempo	Dimensión temporal: fecha, día, mes, año, trimestre, nombre del mes, día de la semana.
dim_cliente	Datos del cliente: nombre, correo, teléfono, ciudad, estado, segmento (Minorista/Mayorista/Corporativo).
dim_producto	Catálogo de productos: nombre, categoría, subcategoría, marca, precio unitario.
dim_region	Zonas geográficas: nombre del estado, país, zona (Norte/Sur/Centro/Occidente).
fact_ventas	Tabla de hechos: cantidad, precio_venta, descuento, monto_total, costo_total, ganancia. Conectada a las 4 dimensiones.

8.2 Creación de tablas en PostgreSQL

Las tablas se crean ejecutando el script DDL incluido en el Jupyter Notebook (Celda 4). Al ejecutarla correctamente, las 5 tablas quedan registradas en la base de datos.

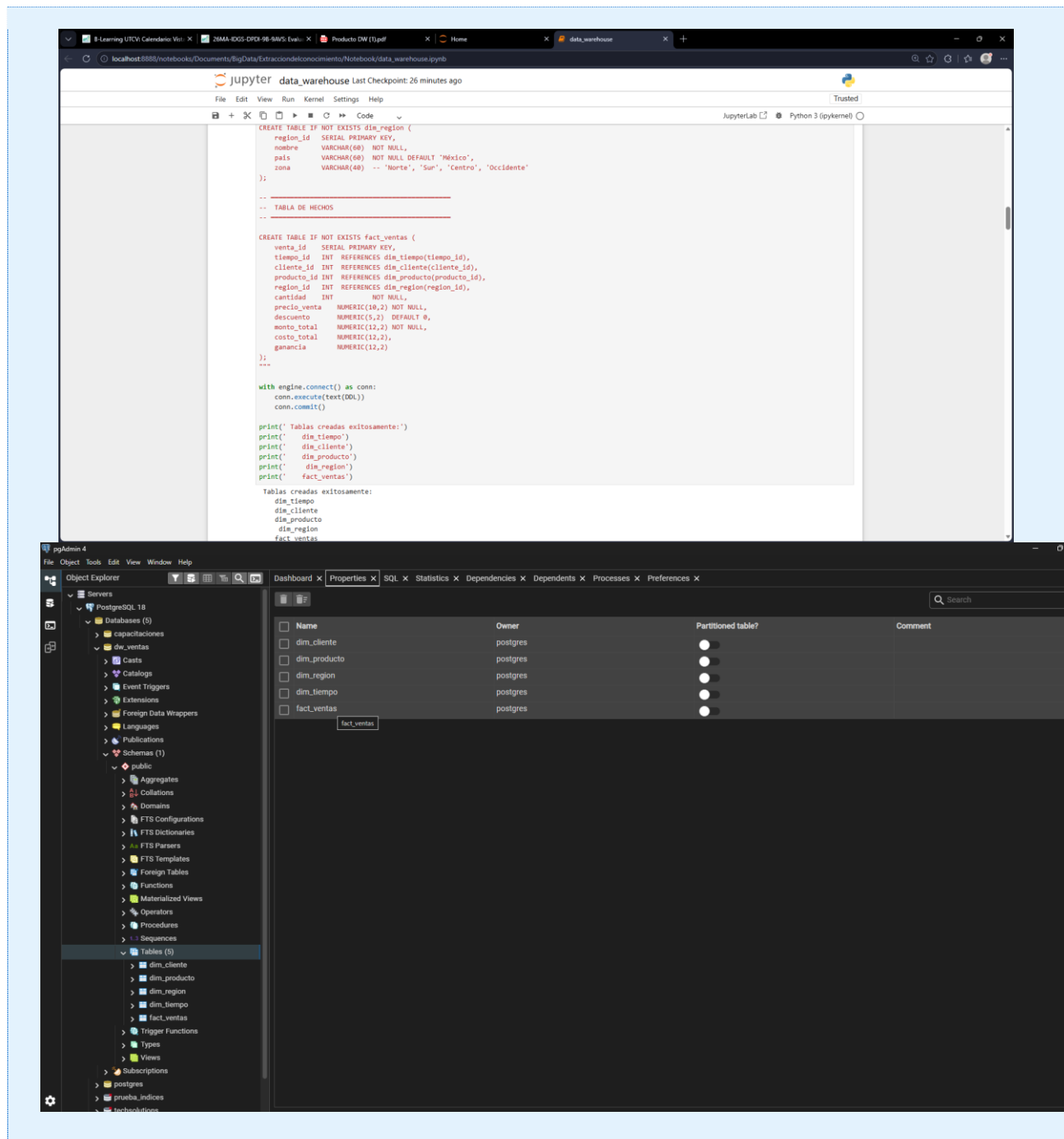


Figura: Tablas de dimensión y hechos creadas en la base de datos dw_ventas.

PARTE III

Proceso ETL — Carga de Datos con Python

9. Proceso ETL con Python

9.1 Instalación del entorno Jupyter y librerías

Se instala Jupyter Notebook o JupyterLab junto con las librerías necesarias. Desde la terminal (Anaconda Prompt o CMD con Python instalado) se ejecuta:

```
pip install jupyter pandas sqlalchemy psycopy2-binary matplotlib seaborn
```

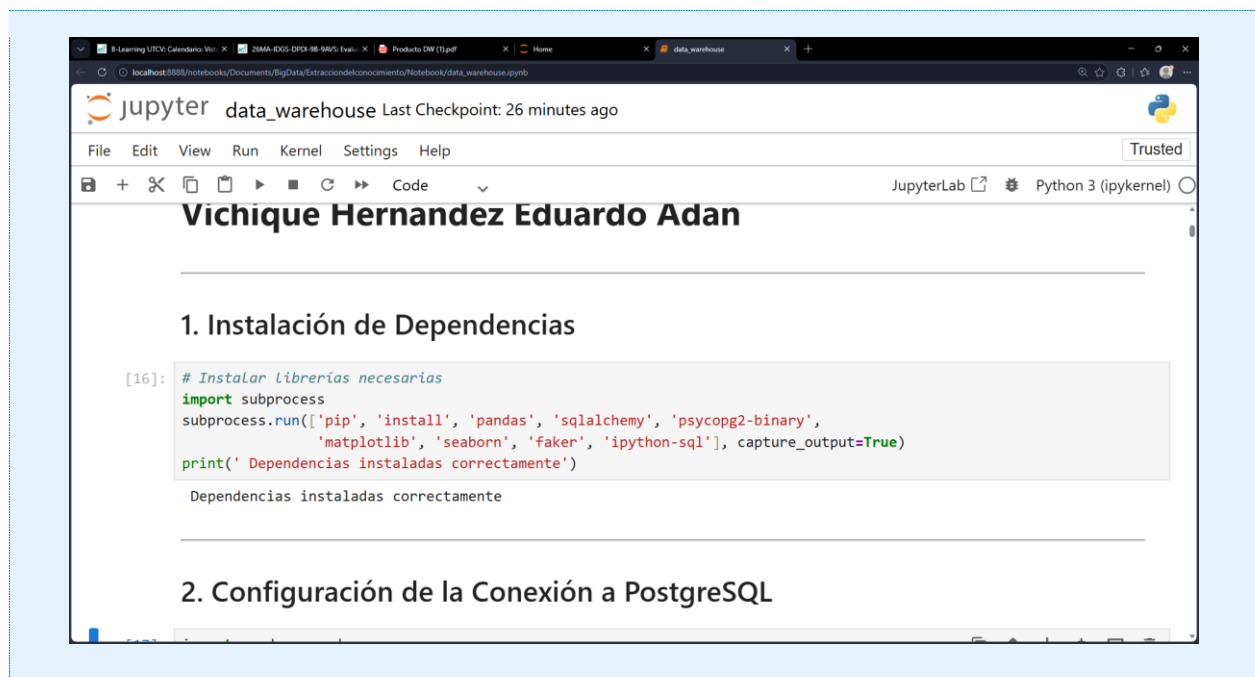


Figura: Jupyter Notebook con el archivo del proyecto Data Warehouse abierto.

9.2 Conexión a PostgreSQL desde Python

La celda de conexión establece el vínculo entre Python y PostgreSQL mediante SQLAlchemy. Al ejecutarse correctamente muestra el mensaje de conexión exitosa.



```
[17]: import pandas as pd
from sqlalchemy import create_engine, text
import warnings
warnings.filterwarnings('ignore')

# --- CONFIGURACIÓN ---
DB_USER = 'postgres'
DB_PASSWORD = '181118'
DB_HOST = 'localhost'
DB_PORT = '5432'
DB_NAME = 'dw_ventas'

# Primero conectar a postgres para crear la base de datos si no existe
engine_default = create_engine(
    f'postgresql+psycopg2://{DB_USER}:{DB_PASSWORD}@{DB_HOST}:{DB_PORT}/{DB_NAME}',
    isolation_level='AUTOCOMMIT'
)

with engine_default.connect() as conn:
    result = conn.execute(text("SELECT 1 FROM pg_database WHERE datname = 'dw_ventas'"))
    if not result.fetchone():
        conn.execute(text('CREATE DATABASE dw_ventas'))
        print(' Base de datos dw_ventas creada')
    else:
        print(' Base de datos dw_ventas ya existe')

# Conectar al DW
```

Figura: Conexión exitosa entre Python/SQLAlchemy y PostgreSQL.

9.3 Generación de archivos CSV

La Celda 5.1 genera los archivos CSV simulados con datos de dimensiones (tiempo, regiones, productos, clientes). Estos archivos se guardan en la carpeta /data del proyecto.

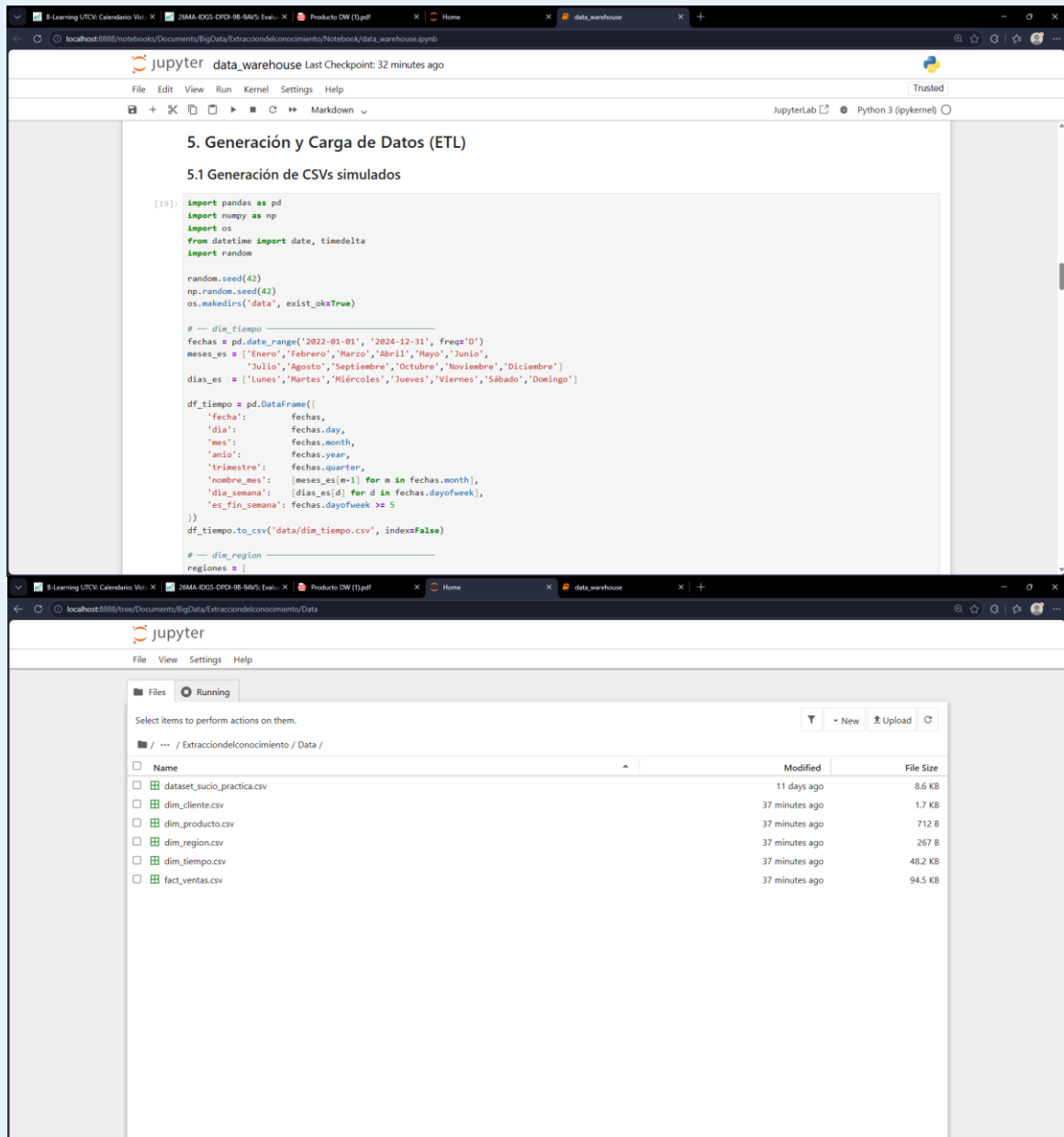


Figura: Archivos CSV generados con datos simulados para el Data Warehouse.

9.4 Carga de dimensiones a PostgreSQL

La Celda 5.2 ejecuta el proceso ETL que carga los datos de los CSVs a las tablas de dimensión en PostgreSQL usando pandas y SQLAlchemy.

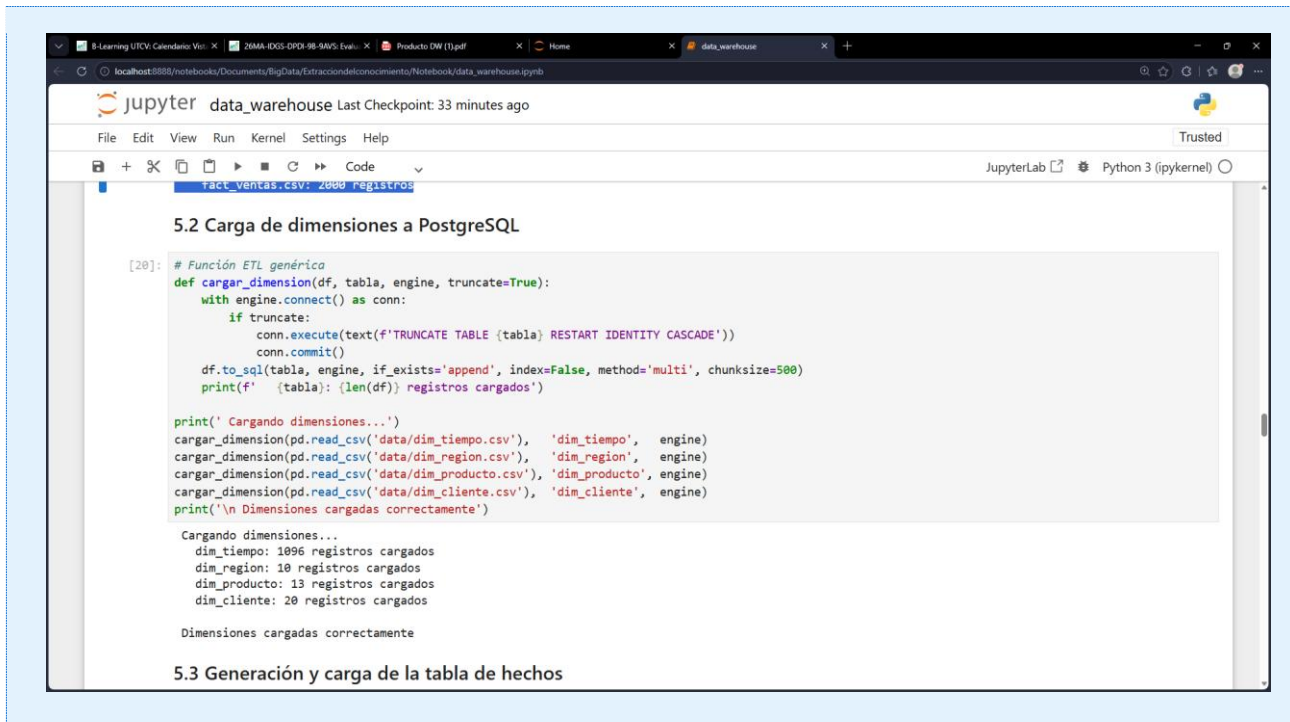


Figura: Dimensiones cargadas exitosamente en PostgreSQL desde archivos CSV.

9.5 Carga de la tabla de hechos

La Celda 5.3 genera y carga 2,000 registros de ventas simuladas a la tabla `fact_ventas`, respetando la integridad referencial con todas las dimensiones.

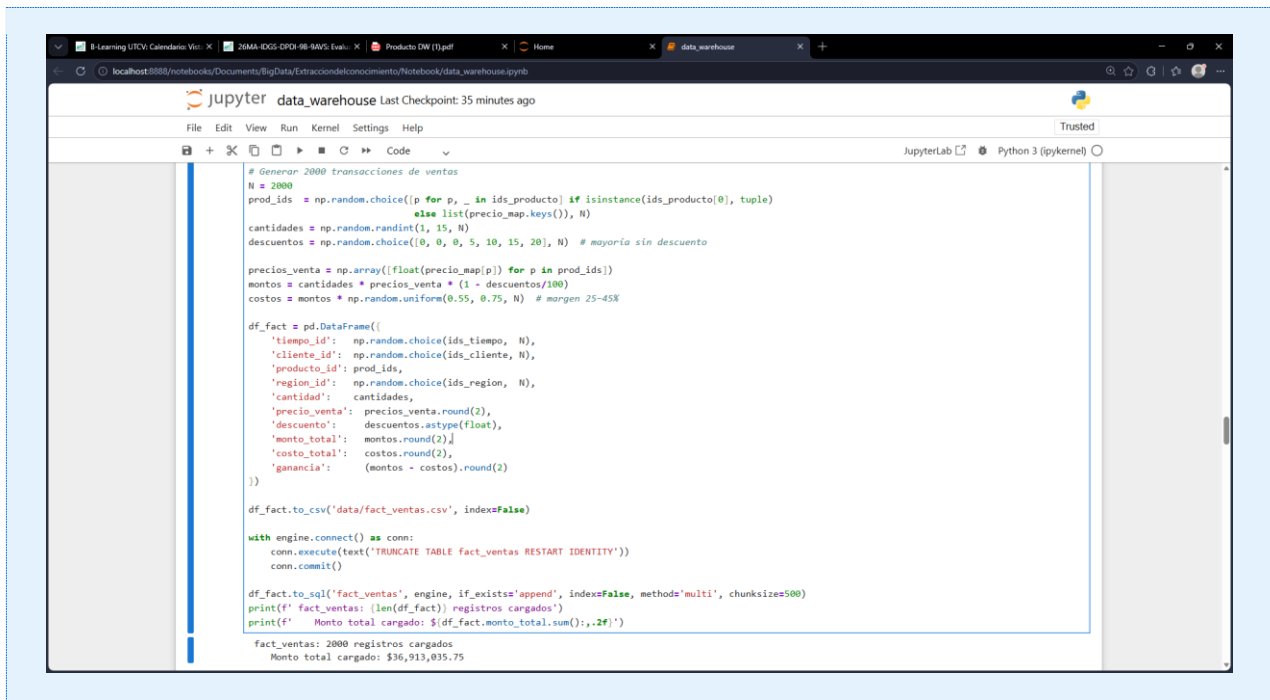


Figura: Tabla de hechos `fact_ventas` cargada con 2,000 transacciones de ventas.

9.6 Verificación en pgAdmin

Tras la carga de datos, se verifica en pgAdmin que las tablas contienen registros ejecutando un `SELECT COUNT(*)` o revisando las propiedades de la tabla.

pgAdmin 4

File Object Tools Edit View Window Help

Dashboard Properties SQL Statistics Dependencies Dependents Processes Preferences public.fact_ventas/dw_ventas/postgres@PostgreSQL 18

Query Query History

```
1 SELECT * FROM public.fact_ventas 2 ORDER BY venta_id ASC
```

Data Output Messages Notifications

Showing rows: 1 to 1000 Page No: 1 of 2

venta_id [PK] integer	tiempo_id integer	cliente_id integer	producto_id integer	region_id integer	cantidad integer	precio_venta numeric (10,2)	descuento numeric (5,2)	monto_total numeric (12,2)	costo_total numeric (12,2)	ganancia numeric (12,2)	
1	1	640	13	7	5	13	4500.00	20.00	4580.00	28045.24	18754.76
2	2	173	19	4	7	5	450.00	5.00	2137.50	1470.89	666.61
3	3	259	7	13	1	5	850.00	0.00	4250.00	2844.76	1405.24
4	4	517	9	11	6	11	120.00	5.00	1254.00	815.50	438.50
5	5	729	19	8	8	4	1900.00	15.00	6460.00	4814.23	1645.77
6	6	523	13	13	1	6	850.00	0.00	5100.00	3502.12	1597.88
7	7	839	8	5	9	3	1200.00	10.00	3240.00	2025.29	1214.71
8	8	117	18	7	3	9	4500.00	0.00	40500.00	28755.59	11744.41
9	9	184	2	10	1	5	95.00	10.00	427.50	310.76	116.74
10	10	162	18	3	1	3	650.00	15.00	1657.50	953.14	704.36
11	11	543	18	7	6	11	4500.00	0.00	49500.00	29210.47	20289.53
12	12	839	11	11	6	1	120.00	15.00	102.00	60.51	41.49
13	13	1090	1	11	1	10	120.00	0.00	1200.00	825.89	374.11

Total rows: 2000 Query complete 00:00:00.170

CRLF Ln 1

Figura: Verificación en pgAdmin: datos cargados correctamente en el Data Warehouse.

PARTE IV

Análisis, Consultas SQL y Visualizaciones

10. Consultas SQL y Análisis

10.1 Ventas totales por año

Se realiza una consulta JOIN entre fact_ventas y dim_tiempo para obtener las ventas totales, número de transacciones y ganancia total agrupadas por año.

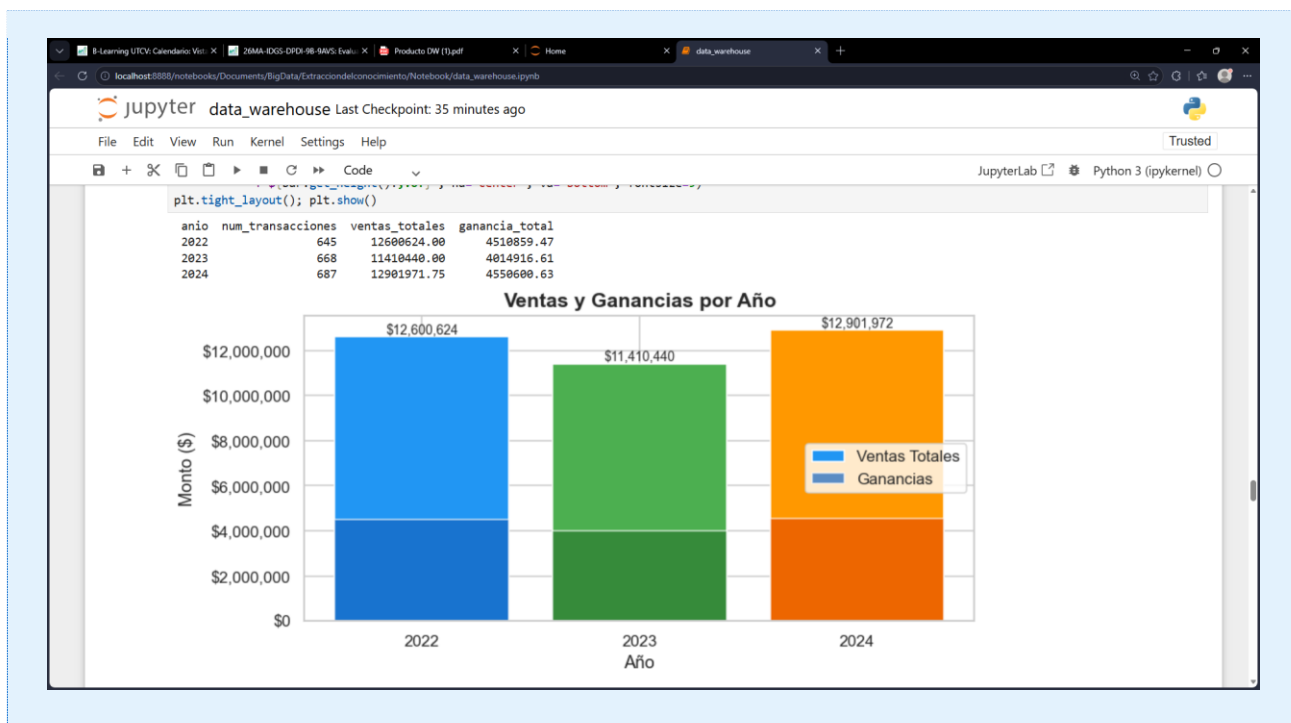


Figura: Ventas y ganancias anuales — gráfica de barras generada con Matplotlib.

10.2 Top 5 productos más vendidos

Consulta que combina fact_ventas con dim_producto para identificar los productos con mayores ingresos y su distribución por categoría.

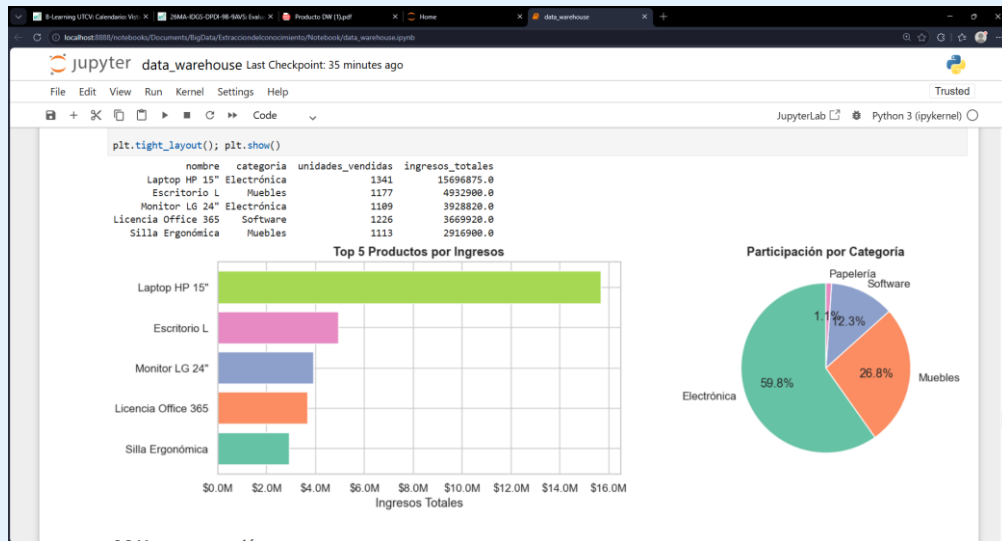


Figura: Top 5 productos por ingresos y participación por categoría — gráficas Matplotlib.

10.3 Ventas por región

Análisis geográfico de ventas y ganancias por estado/región mediante JOIN con dim_region.



Figura: Ventas y ganancias por región geográfica — gráfica de barras agrupadas.

10.4 Tendencia mensual de ventas

Consulta que extrae ventas mensuales por año para visualizar la evolución temporal y detectar patrones de estacionalidad.

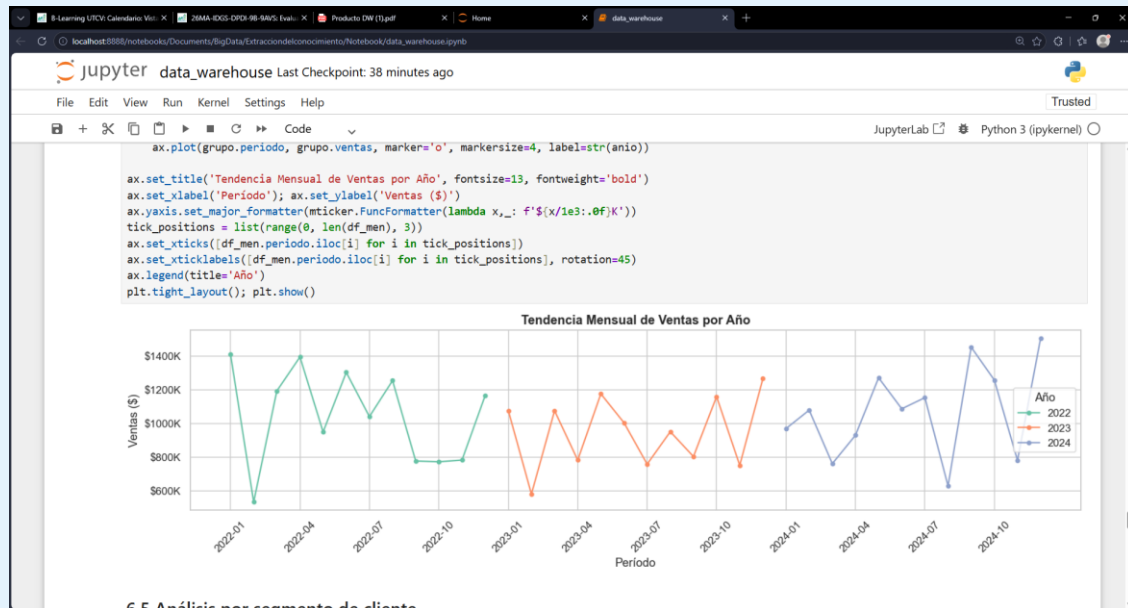


Figura: Tendencia mensual de ventas por año — gráfica de líneas con Matplotlib.

10.5 Análisis por segmento de cliente

Segmentación de clientes en Minorista, Mayorista y Corporativo para comparar ventas totales y margen de ganancia por segmento.

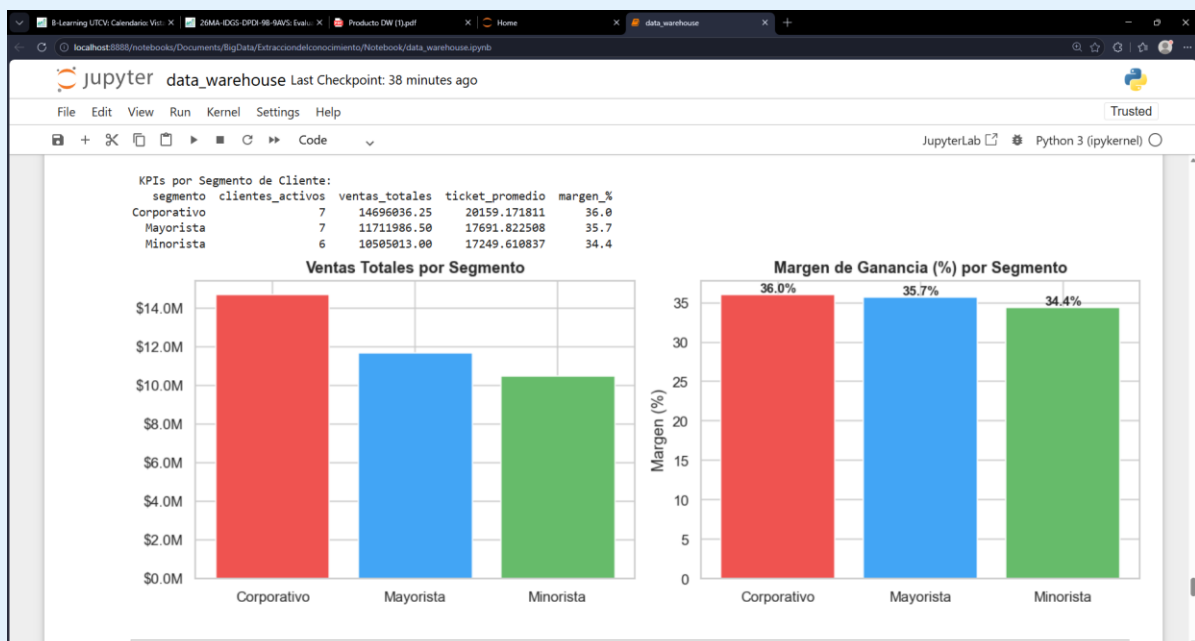


Figura: KPIs y margen de ganancia por segmento de cliente.

10.6 Dashboard de KPIs globales

Resumen ejecutivo con los indicadores clave del negocio: total de transacciones, ventas totales, ganancia, ticket promedio y margen global.

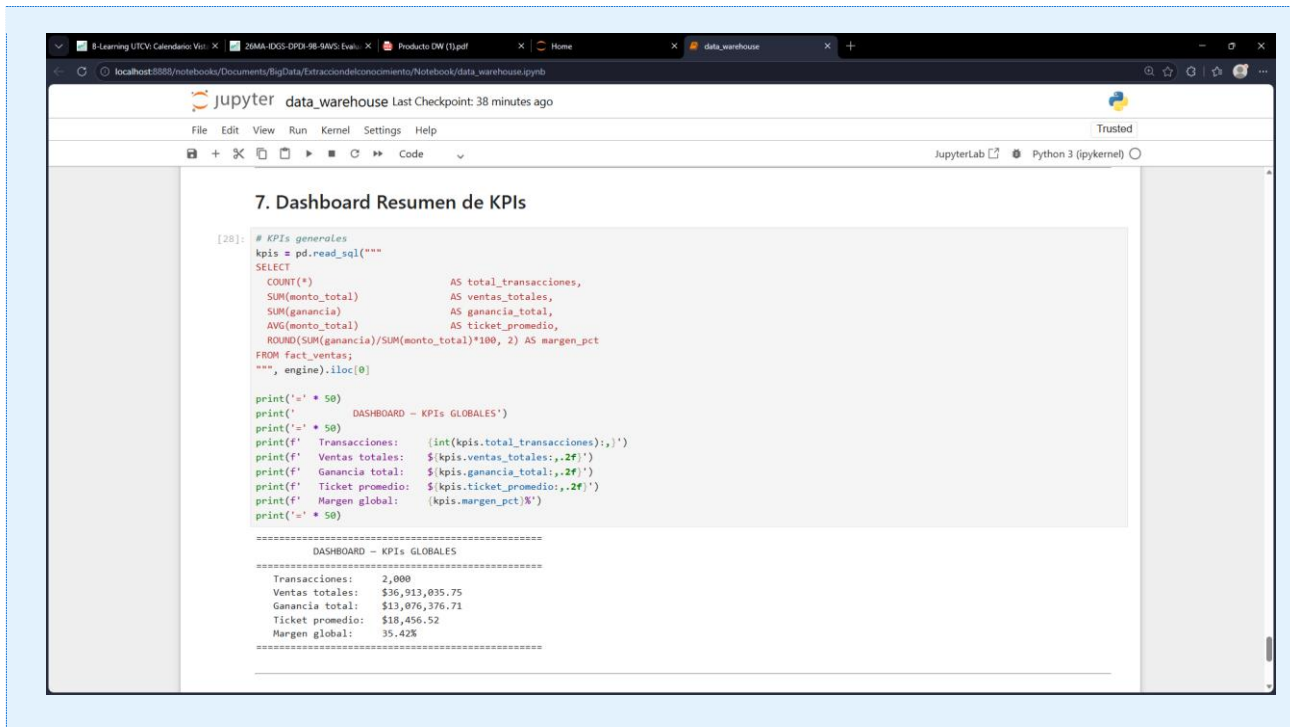


Figura: Dashboard resumen — KPIs globales del Data Warehouse.

PARTE V

Conclusiones

11. Conclusiones

Considero que este proyecto fue una experiencia muy enriquecedora porque me permitió aplicar conocimientos teóricos en un caso práctico y desarrollar nuevas habilidades relacionadas con bases de datos, programación y análisis de información. Además, me ayudó a entender que un Data Warehouse no solo sirve para almacenar grandes cantidades de datos, sino para convertirlos en conocimiento que apoye la toma de decisiones. En el futuro me gustaría seguir aprendiendo sobre inteligencia de negocios, análisis de datos y herramientas más avanzadas que permitan aprovechar aún más el valor de la información dentro de las organizaciones.

12. Referencias

- Inmon, W. H. (2005). Building the Data Warehouse (4th ed.). Wiley Publishing.
- Kimball, R., & Ross, M. (2013). The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling (3rd ed.). Wiley.
- PostgreSQL Global Development Group. (2024). PostgreSQL 16 Documentation. Recuperado de <https://www.postgresql.org/docs/>
- McKinney, W. (2022). Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter (3rd ed.). O'Reilly Media.
- SQLAlchemy Documentation. (2024). SQLAlchemy 2.0. Recuperado de <https://docs.sqlalchemy.org/>
- Jupyter Project. (2024). Jupyter Notebook Documentation. Recuperado de <https://jupyter-notebook.readthedocs.io/>
- Hunter, J. D., et al. (2024). Matplotlib: Visualization with Python. Recuperado de <https://matplotlib.org/>